ARCHITECTURE DESIGN DOCUMENT | PHASE 2

End-to-End Data Analytics Platform

Microsoft Fabric | Public Data | Spain Indicators

Complete architecture design covering system topology, data flow, naming conventions, object inventory, and operational decisions for the end-to-end data analytics platform built on Microsoft Fabric.

AUTHOR

Ssr Data Analyst

6 years experience

Microsoft Stack

STACK

Microsoft Fabric

Lakehouse | Warehouse

PySpark | Power BI

DATE

April 2026

Duration: 1-2 months

Version 1.1

1. Executive Summary

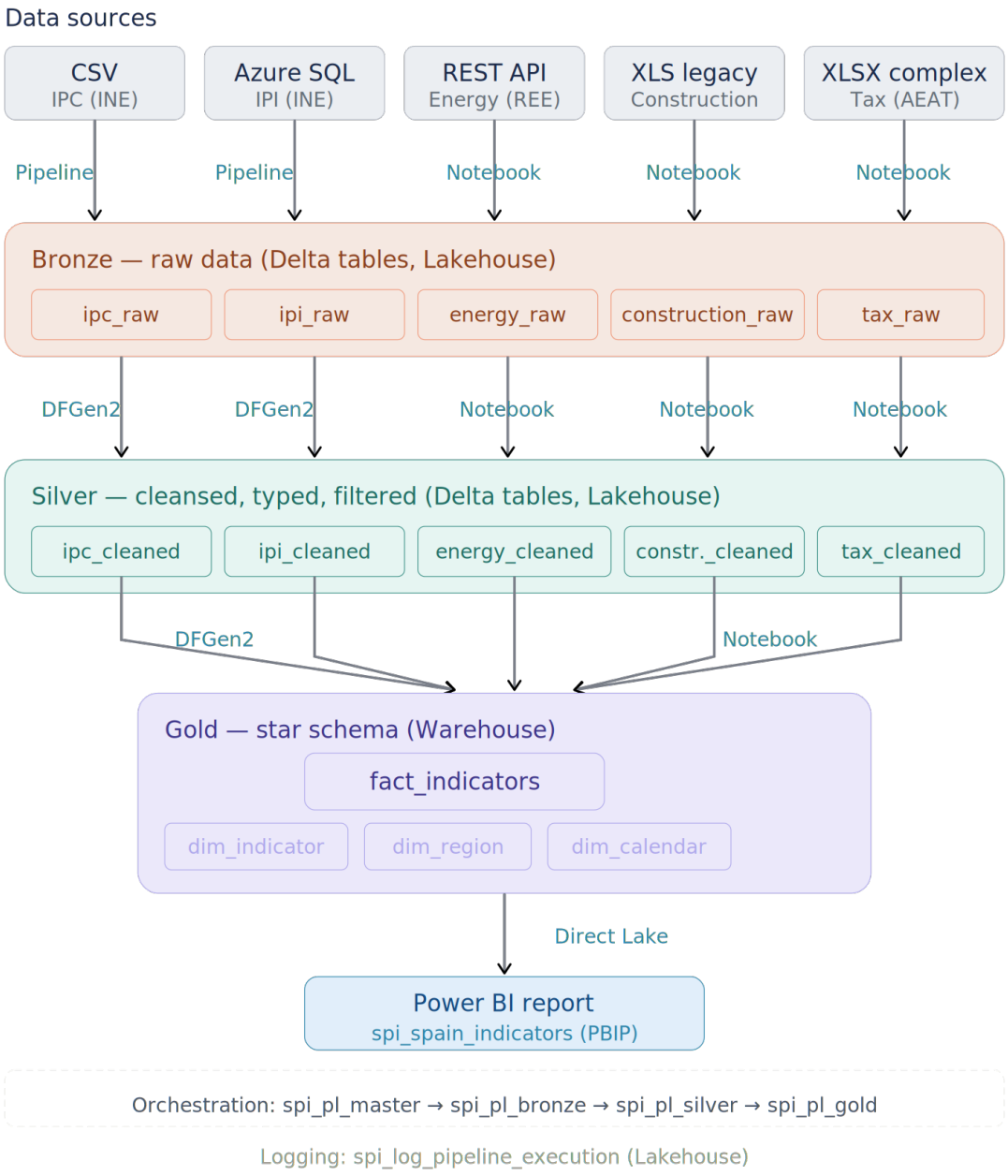
This document defines the architecture design for the End-to-End Data Analytics Platform. It translates the requirements and source catalog defined in Phase 1 into a concrete system design: how data flows through the platform, which Microsoft Fabric components handle each step, how objects are named and organized, and how the solution operates.

The architecture follows a **Bronze-Silver-Gold medallion pattern** implemented across Fabric Lakehouse (Bronze and Silver layers) and Fabric Warehouse (Gold layer), with Power BI consuming the analytical model via Direct Lake connectivity. Orchestration is handled by layered Data Pipelines, and error handling is built into the design with a centralized logging table.

> *All decisions documented here are justified by the nature of the data, the capabilities of each Fabric component, and the strategic goal of building a professional freelance portfolio.*

2. High-Level Architecture

The consolidated architecture diagram below shows the complete data flow from the five source systems through the medallion layers to the Power BI report.



Architecture Overview

Five public data sources (CSV, Azure SQL Database, REST API, XLS legacy, and XLSX complex) are ingested into the Bronze layer of a Fabric Lakehouse as raw Delta tables. Each source is then cleansed, typed, and filtered in the Silver layer. Finally, all five sources converge into a unified star schema in the Fabric Warehouse (Gold layer), which is consumed by a Power BI report via Direct Lake connectivity.

Key Design Principles

• **Medallion architecture (Bronze → Silver → Gold):** separates raw ingestion, data cleansing, and dimensional modeling into distinct layers with clear responsibilities.

• **Component-appropriate tooling:** Data Pipelines and Dataflows Gen2 handle simple sources; Notebooks handle complex sources. Nothing is forced to demonstrate tool usage.

• **Unified analytical model:** all five sources converge into a single fact table, enabling cross-domain analysis and ensuring the data warehouse is scalable.

• **Layered orchestration:** a master pipeline delegates to layer-specific pipelines (Bronze, Silver, Gold), enabling independent re-execution per layer.

3. Medallion Architecture Design

3.1 Layer Definitions

The medallion architecture separates the data lifecycle into three layers, each with a distinct purpose and storage location.

Layer	Purpose	Storage	Format
Bronze	Raw data as received from source systems. Minimal processing: encoding resolution and metadata addition.	Lakehouse (OneLake)	Delta tables
Silver	Cleansed, typed, filtered, and normalized data. Flat tabular structure, not yet dimensionally modeled.	Lakehouse (OneLake)	Delta tables
Gold	Dimensionally modeled star schema optimized for analytical querying and Power BI consumption.	Warehouse	Warehouse tables

3.2 What Happens in Each Layer

Bronze (raw ingestion):

- Data arrives as-is from the source system.
- Encoding issues are resolved during ingestion (e.g., ISO-8859-15 → UTF-8 for INE CSVs).
- Metadata columns are added: `_ingestion_timestamp` and `_source_file` (or `_source_table` for SQL sources).
- No business logic, no filtering, no transformations.

Silver (cleansing and normalization):

- Period parsing (e.g., `2026M02` → `year` + `month` columns).
- Geographic filtering: only Nacional, Cataluña, and Madrid are retained.
- Column renaming to snake_case with descriptive English names.
- Data type enforcement (indices as decimal, dates as proper types).
- Source-specific cleansing: removal of subtotal rows (XLS), JSON flattening (API), unpivot operations (XLSX), deduplication.
- Output is a clean, flat table — not dimensionally modeled.

Gold (dimensional modeling):

- Silver tables are mapped to the unified star schema.
- Foreign keys are assigned to all five dimension tables.
- Derived measures (YoY, MoM, YTD, QTD) are calculated and stored as additional rows via dim_measure.
- Output is a star schema optimized for Power BI Direct Lake.

3.3 Why Silver Exists

The original on-premise architecture (SSIS) used a two-layer approach: Stage (raw) → Data Warehouse (star schema). This worked well but combined data cleansing and dimensional modeling in a single step.

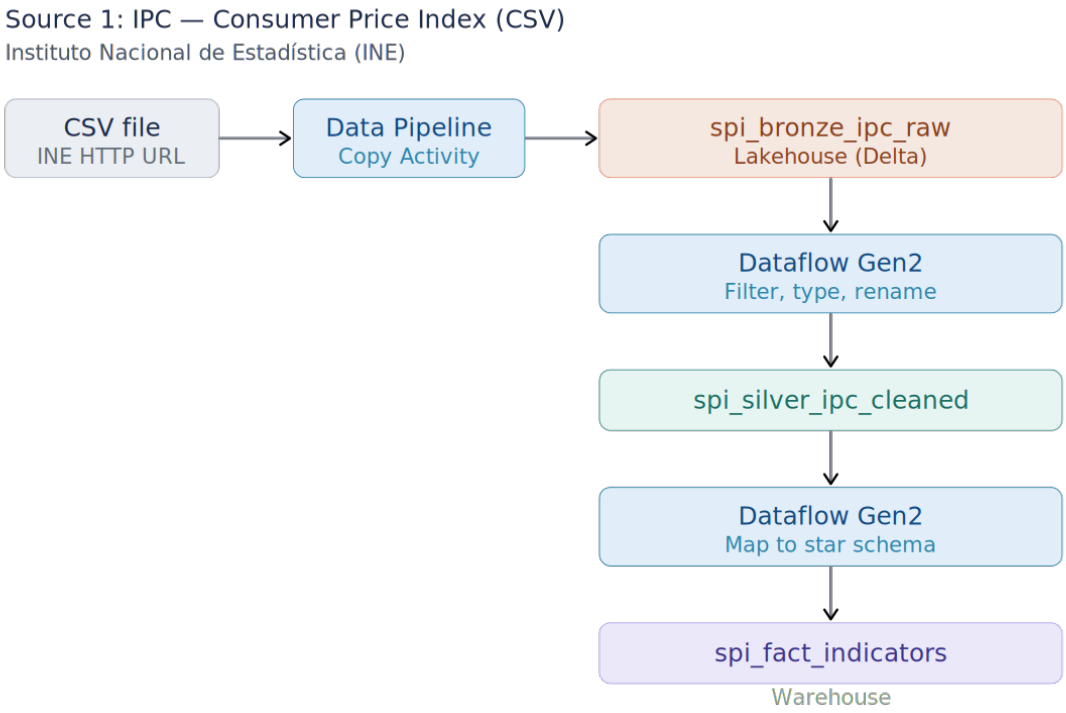
The Silver layer separates these concerns, providing three advantages:

- **Debugging:** when an issue appears in Gold, Silver provides a checkpoint to determine whether the problem originated in the source data or in the modeling logic.
- **Reusability:** clean, flat data in Silver can be consumed by other workloads (exploratory analysis, ad-hoc queries) without requiring the dimensional model.
- **Industry alignment:** Bronze-Silver-Gold is the standard medallion pattern in cloud data platforms. Implementing it correctly demonstrates modern data engineering practices.

4. Data Flow Diagrams — Individual Sources

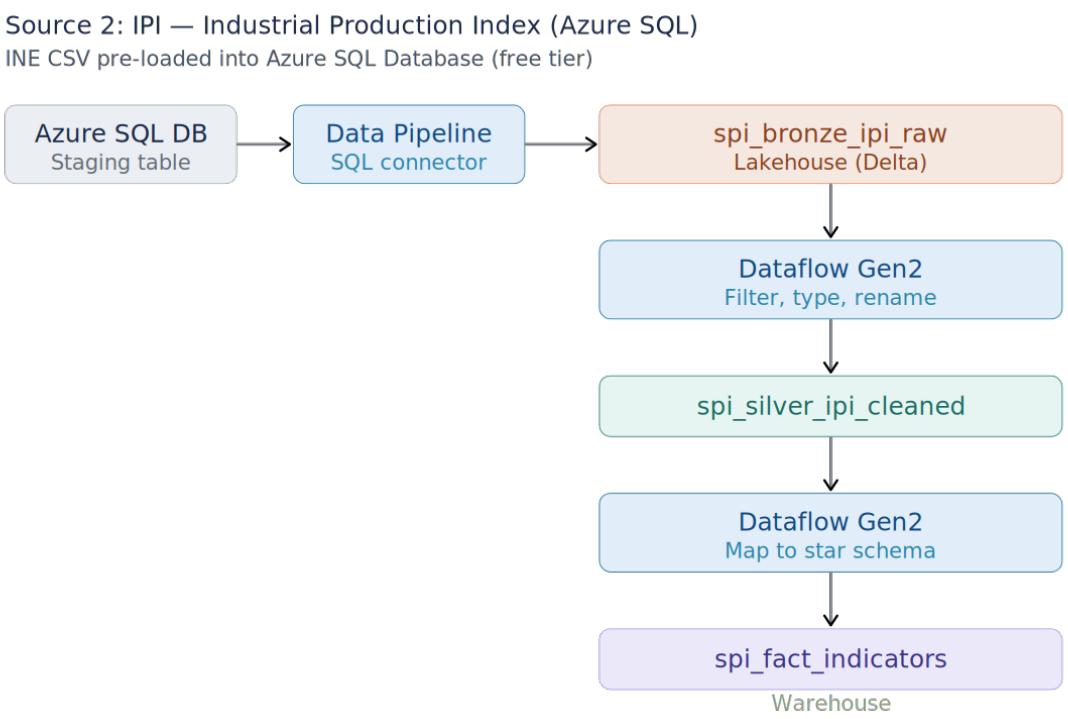
Each source follows a specific data flow path through the medallion layers. The diagrams below show the Fabric components involved at each step.

4.1 Source 1: IPC — Consumer Price Index (CSV)



Step	Component	Description
Ingestion	Data Pipeline (Copy Activity)	Downloads CSV from INE HTTP URL into Bronze
Bronze → Silver	Dataflow Gen2	Filters geographic scope, parses period, renames columns, enforces types
Silver → Gold	Dataflow Gen2	Maps cleaned data to fact_indicators star schema

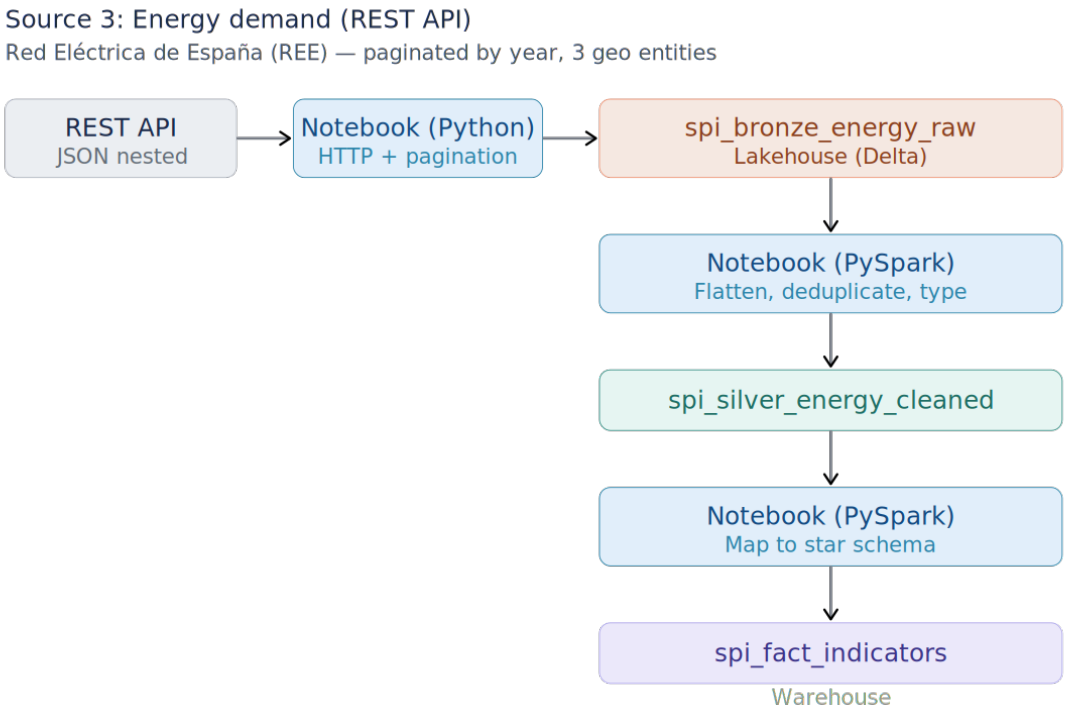
4.2 Source 2: IPI — Industrial Production Index (Azure SQL)



Step	Component	Description
Ingestion	Data Pipeline (Copy Activity, SQL connector)	Reads from Azure SQL Database staging table into Bronze
Bronze → Silver	Dataflow Gen2	Filters geographic scope, parses period, renames columns, enforces types
Silver → Gold	Dataflow Gen2	Maps cleaned data to fact_indicators star schema

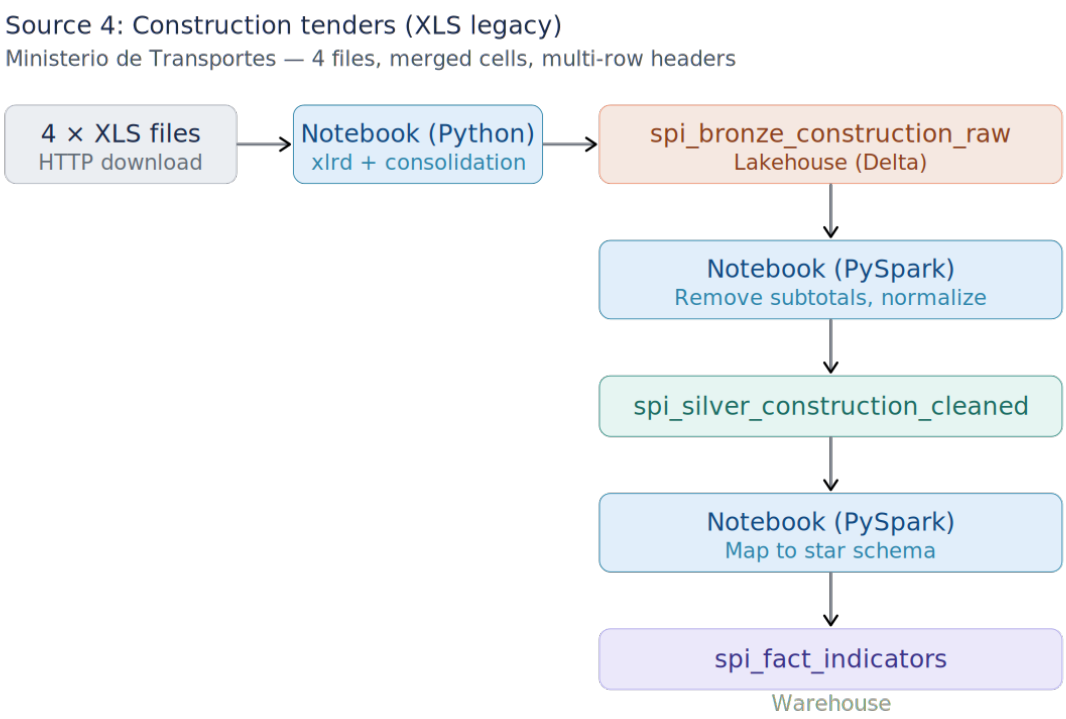
Note: the Azure SQL Database setup (provisioning, CSV bulk insert) is documented as a prerequisite in Phase 4, not as part of the Fabric pipeline.

4.3 Source 3: Energy Demand (REST API)



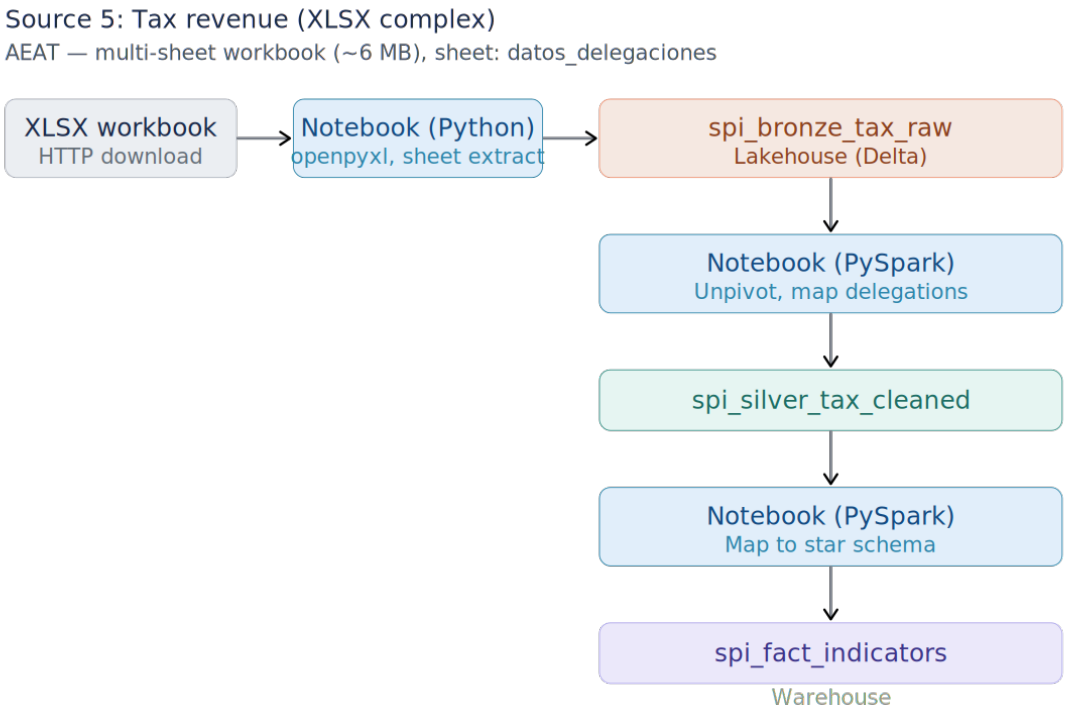
Step	Component	Description
Ingestion	Notebook (Python)	Calls REE API with pagination by year, 3 geographic entities, parses nested JSON
Bronze → Silver	Notebook (PySpark)	Flattens nested structure, deduplicates, enforces types
Silver → Gold	Notebook (PySpark)	Maps cleaned data to fact_indicators star schema

4.4 Source 4: Construction Tenders (XLS Legacy)



Step	Component	Description
Ingestion	Notebook (Python, xlrd)	Downloads and parses 4 XLS files with merged cells and multi-row headers, consolidates into single table
Bronze → Silver	Notebook (PySpark)	Removes subtotal rows, normalizes column names, enforces types
Silver → Gold	Notebook (PySpark)	Maps cleaned data to fact_indicators star schema

4.5 Source 5: Tax Revenue (XLSX Complex)



Step	Component	Description
Ingestion	Notebook (Python, openpyxl)	Extracts datos_delegaciones sheet from ~6 MB workbook
Bronze → Silver	Notebook (PySpark)	Unpivots crosstab format, maps 56 delegations to 3 geographic entities, enforces types
Silver → Gold	Notebook (PySpark)	Maps cleaned data to fact_indicators star schema

5. Component Mapping

The following matrix provides a consolidated view of which Fabric component handles each step for each source.

#	Source	Format	Bronze Ingestion	Silver Transform	Gold Load	Complexity
1	IPC (INE)	CSV	Pipeline (Copy Activity)	Dataflow Gen2	Dataflow Gen2	Low
2	IPI (INE)	CSV → Azure SQL	Pipeline (SQL Connector)	Dataflow Gen2	Dataflow Gen2	Medium
3	Energy (REE)	REST API (JSON)	Notebook (Python)	Notebook (PySpark)	Notebook (PySpark)	Medium-High
4	Construction (MITMA)	XLS (Legacy)	Notebook (Python)	Notebook (PySpark)	Notebook (PySpark)	High
5	Tax Revenue (AEAT)	XLSX (Complex)	Notebook (Python)	Notebook (PySpark)	Notebook (PySpark)	High

Design principle: *Dataflows Gen2 handle simple, low-complexity transformations (Sources 1–2) where Spark overhead is unnecessary. Notebooks handle complex ingestion and transformation (Sources 3–5) where programmatic control is required. Data Pipelines orchestrate the entire flow.*

6. Data Model — Gold Layer (Star Schema)

The Gold layer implements a unified star schema in the Fabric Warehouse, following the same proven pattern used in the author's production project for a Spanish public entity (80+ sources, star schema, Power BI + web consumption).

6.1 Design Decision: Unified Fact Table

All five sources converge into a single fact table (`spi_fact_indicators`). This decision is driven by three factors:

- **Scalability:** adding a new source requires a new pipeline that produces rows in the existing fact table — no schema changes, no new tables.
- **Cross-domain analysis:** a unified model enables correlation across domains (e.g., energy demand trends vs. industrial production) through a single Power BI report with shared filters.
- **Proven pattern:** this architecture has been validated in a production environment with 80+ sources over two years.

6.2 Star Schema Overview

Fact Table

Table	Grain	Key Columns
spi_fact_indicators	One row per indicator × region × period × measure × source	5 FKs (indicator_key, region_key, calendar_key, measure_key, source_key) + indicator_value

Dimension Tables

Table	Description	Key Attributes
spi_dim_indicator	Indicator metadata and classification	indicator_name, indicator_category, domain, unit_of_measure, frequency
spi_dim_region	Geographic entities	region_key, region_code, region_name, region_type
spi_dim_calendar	Time dimension (monthly granularity)	calendar_key (YYYYMM), year, month, month_name, quarter, year_month, is_current_year
spi_dim_measure	Measure type classification	measure_code (IDX, YoY, MoM, YTD, QTD), measure_name, measure_description
spi_dim_source	Data source metadata	source_code, source_name, source_institution, source_domain, source_url, update_frequency

Note: Derived measures (YoY, MoM, YTD, QTD) are modeled as separate rows in the fact table via `spi_dim_measure`, not as additional columns. This provides a cleaner, more scalable design that avoids wide fact tables and enables flexible measure selection in Power BI.

Detailed column specifications, data types, and business rules are delivered in Phase 3 (Functional Specification) and Phase 4 (Technical Specification).

7. Object Inventory

7.1 Lakehouse Objects — Bronze

Table	Source	Metadata Columns
spi_bronze_ipc_raw	CSV — IPC (INE)	_ingestion_timestamp, _source_file
spi_bronze_ipi_raw	Azure SQL — IPI (INE)	_ingestion_timestamp, _source_table
spi_bronze_energy_raw	REST API — Energy (REE)	_ingestion_timestamp, _source_file
spi_bronze_construction_raw	XLS × 4 — Construction (MITMA)	_ingestion_timestamp, _source_file
spi_bronze_tax_raw	XLSX — Tax Revenue (AEAT)	_ingestion_timestamp, _source_file

7.2 Lakehouse Objects — Silver

Table	Source
spi_silver_ipc_cleaned	Bronze IPC
spi_silver_ipi_cleaned	Bronze IPI
spi_silver_energy_cleaned	Bronze Energy
spi_silver_construction_cleaned	Bronze Construction
spi_silver_tax_cleaned	Bronze Tax

7.3 Warehouse Objects — Gold

Table	Type
spi_fact_indicators	Fact table
spi_dim_indicator	Dimension
spi_dim_region	Dimension
spi_dim_calendar	Dimension
spi_dim_measure	Dimension
spi_dim_source	Dimension

7.4 Operational Objects

Table	Location	Purpose
spi_log_pipeline_execution	Lakehouse	Centralized execution log for all pipeline runs

8. Naming Conventions

8.1 General Rules

Rule	Convention	Example
Global prefix	spi_	spi_bronze_ipc_raw
Tables and columns	snake_case	indicator_value, region_name
Workspaces	kebab-case	spi-spain-indicators-dev
Bronze metadata columns	Underscore prefix	_ingestion_timestamp

8.2 Fabric Objects

Workspaces

Object	Name
Workspace DEV	spi-spain-indicators-dev
Workspace PROD	spi-spain-indicators-prod

Storage

Object	Name
Lakehouse	spi_lakehouse
Warehouse	spi_warehouse

Notebooks

Notebook	Layer	Source
spi_nb_bronze_energy	Bronze	REST API ingestion
spi_nb_bronze_construction	Bronze	XLS ingestion + parsing
spi_nb_bronze_tax	Bronze	XLSX ingestion
spi_nb_silver_energy	Silver	Flatten + deduplicate
spi_nb_silver_construction	Silver	Remove subtotals + normalize
spi_nb_silver_tax	Silver	Unpivot + delegation mapping
spi_nb_gold_load	Gold	Silver → Warehouse load (sources 3, 4, 5) — detail in Phase 4

Dataflows Gen2

Dataflow	Layer	Source
spi_df_silver_ipc	Silver	IPC cleansing
spi_df_silver_ipi	Silver	IPI cleansing
spi_df_gold_ipc	Gold	IPC → Warehouse load
spi_df_gold_ipi	Gold	IPI → Warehouse load
spi_df_gold_dimensions	Gold	Dimension table generation

Data Pipelines

Pipeline	Purpose	Internal Activities
spi_pl_master	Master orchestrator — executes Bronze → Silver → Gold in sequence	Invokes the 3 layer pipelines
spi_pl_bronze	Ingests all 5 sources into Bronze	2 Copy Activities (IPC, IPI) + 3 Notebook Activities (Energy, Construction, Tax)
spi_pl_silver	Transforms Bronze → Silver for all 5 sources	2 Dataflow Activities (IPC, IPI) + 3 Notebook Activities (Energy, Construction, Tax)
spi_pl_gold	Loads Silver → Warehouse for all sources	Dataflows (IPC, IPI, Dimensions) + Notebook (Sources 3, 4, 5) — detail in Phase 4

Power BI (PBIP format)

Object	Name
Semantic Model + Report	spi_spain_indicators

8.3 Abbreviation Reference

Abbreviation	Meaning
nb	Notebook
pl	Pipeline
df	Dataflow Gen2
sm	Semantic Model
rpt	Report

9. Loading Strategy

9.1 Strategy by Source

#	Source	Strategy	Rationale
1	IPC (CSV)	Truncate and reload	CSV contains full historical dataset; each download replaces the previous one
2	IPI (Azure SQL)	Truncate and reload	Same as IPC — full dataset available in staging table
3	Energy (API)	Initial full load + incremental	API has date range constraints; initial load covers full history, subsequent loads fetch recent periods with N-period overlap
4	Construction (XLS)	Truncate and reload	XLS files contain full historical dataset
5	Tax Revenue (XLSX)	Truncate and reload	XLSX workbook contains full historical dataset

9.2 Layer Behavior

Layer	Behavior
Bronze	Truncate and reload per source (except API: incremental append after initial load)
Silver	Truncate and reload — regenerated entirely from Bronze
Gold	Truncate and reload — fact and dimension tables rebuilt from Silver

9.3 Production Note

Full refresh strategy is chosen given portfolio scope. In a production environment, this would be replaced by incremental merge/upsert patterns for large datasets, with change detection and watermark-based loading.

10. Environment Strategy (DEV / PROD)

10.1 Workspace Structure

Workspace	Name	Data	Purpose
DEV	spi-spain-indicators-dev	Last 24 periods	Active development, testing, iteration
PROD	spi-spain-indicators-prod	Full historical dataset	Final deployment, screenshots, portfolio

10.2 Deployment Strategy

PROD workspace is created only after development stabilizes in DEV. Artifacts (notebooks, pipelines, dataflows, semantic model, report) are promoted from DEV to PROD via Fabric Deployment Pipelines. Data is not promoted — PROD generates its own data by executing the pipelines with full historical parameters.

10.3 Parametrization

Pipelines accept parameters that control the data scope (date ranges). In DEV, parameters are set to the last 24 periods for fast iteration. In PROD, parameters are set to the full available history per source. Parametrization detail is specified in Phase 4.

10.4 Capacity Constraint

The development environment uses a Fabric Trial Capacity (F4, with potential extension to F64). Data volume in PROD may be adjusted to capacity constraints if needed. This is documented transparently as a trial limitation, not hidden.

11. Error Handling & Logging

11.1 Pipeline Failure Behavior

Scenario	Behavior	Rationale
One source fails within a layer	Continue on error — remaining sources proceed	Sources are independent; a failure in the energy API should not prevent IPC from updating
A layer fails completely	Downstream layers do not execute for affected sources	No point in processing Silver if Bronze didn't complete for that source

11.2 Logging Table

All pipeline executions are logged to a centralized table in the Lakehouse: `spi_log_pipeline_execution`

Column	Description
pipeline_name	Name of the pipeline or notebook that executed
layer	bronze, silver, or gold
source	ipc, ipi, energy, construction, tax, or all
status	success, failed, or running
start_time	Execution start timestamp
end_time	Execution end timestamp
error_message	Error details (null if successful)
rows_processed	Number of rows written (null if failed)

Each notebook and pipeline writes a row at the start of execution (status: running) and updates it upon completion (status: success or failed).

11.3 Retry Policy

Retry is configured only for the REST API source (Energy Demand — REE), which is the only source susceptible to transient failures (timeouts, temporary service unavailability). Configuration: 3 attempts with 30-second intervals, using the native retry policy of Data Pipeline activities.

> *File-based sources (CSV, XLS, XLSX) and Azure SQL do not use retry — if they fail, the issue is structural (file not found, database unavailable) and a retry would not resolve it.*

12. Power BI Format Decision

12.1 PBIP (Power BI Project)

The Power BI deliverable uses the **PBIP format** instead of the traditional PBIX. Starting January 2026, Microsoft has made PBIR (Enhanced Report Format) the default for all new reports, and PBIP is the project structure that enables Git-based version control.

12.2 Rationale

- **Version control:** PBIP saves semantic model and report definitions as text-based files (JSON/TMDL) that Git can track, diff, and merge.
- **Portfolio alignment:** the GitHub repository shows real diffs of DAX measures, relationships, and report pages — not a binary blob.
- **Modern standard:** PBIP represents the current direction of Power BI development practices.

12.3 Repository Structure

```
/src/power-bi/
■■■ spi_spain_indicators.Report/
■■■ spi_spain_indicators.SemanticModel/
■■■ .gitignore
■■■ spi_spain_indicators.pbip
```

13. Key Decisions Summary

#	Decision	Rationale
1	Medallion architecture (Bronze-Silver-Gold)	Industry standard, separates concerns, adds debugging and reusability layer
2	Unified fact table (spi_fact_indicators)	Scalable — new sources add rows, not tables. Proven with 80+ sources
3	dim_source as standalone dimension (5 rows)	Institutional metadata (INE, REE, MITMA, AEAT) justifies dedicated dimension for report navigation and transparency
4	dim_measure for derived calculations (IDX, YoY, etc.)	Measures as rows (not columns) avoids wide fact table; enables flexible measure selection in Power BI
5	Dataflows Gen2 for simple, Notebooks for complex	Matches transformation complexity to tool capability
6	Layered pipeline orchestration (master → bronze/silver/gold)	Enables independent re-execution per layer
7	Continue on error within layers	Sources are independent; one failure should not block the rest
8	Centralized logging table	Operational visibility; demonstrates production-ready thinking
9	PBIP format for Power BI	Enables Git version control of the semantic model and report
10	Two workspaces (DEV/PROD) with Deployment Pipelines	Separates development sandbox from clean portfolio presentation
11	Truncate and reload (except API: incremental)	Appropriate for portfolio scope; production alternative documented
12	Retry policy only for REST API	Only source susceptible to transient failures

14. Next Steps

With the architecture design formalized, the project proceeds to **Phase 3: Functional Specification**, which will deliver:

- Detailed column specifications for all Bronze, Silver, and Gold tables.
- Business rules for each transformation (what each field means, how derived fields are calculated).
- Dimension table content (indicator catalog, region catalog, calendar generation rules).
- Power BI report specification (pages, visuals, filters, measures).
- Data quality rules and validation criteria.

Phase Progress

0	**Context & Strategy** Deliverable: Project Brief (PDF)	COMPLETE
1	**Requirements & Source Catalog** Deliverable: Requirements Document + Source Catalog	COMPLETE
2	**Architecture Design** Deliverable: Architecture Design Document	COMPLETE
3	**Functional Specification** Deliverable: Functional Document (PDF)	NEXT
4	**Technical Specification** Deliverable: Technical Document (PDF)	PENDING
5	**Development in Fabric** Deliverable: Working Solution + Code	PENDING
6	**GitHub Publication** Deliverable: GitHub Repository + README	PENDING

Document version 1.1 | April 2026 | Phase 2 deliverable (updated per Phase 3 alignment)